Module 10

Introduction to SQL/3

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Completed the understanding of basic query structure

Module Objectives

Goals

- To familiarize with set operations, null values and aggregation

Module Outline

- Set Operations: `union`, `intersect`, `except`

Module Outline

- Set Operations: `union`, `intersect`, `except`
- Null Values

Module Outline

- Set Operations: `union`, `intersect`, `except`
- Null Values
- Aggregate Functions: `avg`, `min`, `max`, `sum`, and `count`

Module Outline

- Set Operations: `union`, `intersect`, `except`
- Null Values
- Aggregate Functions: `avg`, `min`, `max`, `sum`, and `count`
- `Group By`

Module Outline

- Set Operations: `union`, `intersect`, `except`
- Null Values
- Aggregate Functions: `avg`, `min`, `max`, `sum`, and `count`
- `Group By`
- `Having`

Module Outline

- Set Operations: `union`, `intersect`, `except`
- Null Values
- Aggregate Functions: `avg`, `min`, `max`, `sum`, and `count`
- `Group By`
- `Having`
- Null Values

Set Operations

Set Operations

- Find courses that ran in Fall 2009 or in Spring 2010

```
(select course_id from section where sem = 'Fall' and year =
    2009)
union
(select course_id from section where sem = 'Spring' and year =
    2010);
```

- Find courses that ran in Fall 2009 and in Spring 2010

```
(select course_id from section where sem = 'Fall' and year =
    2009)
intersect
(select course_id from section where sem = 'Spring' and year =
    2010);
```

- Find courses that ran in Fall 2009 but not in Spring 2010

```
(select course_id from section where sem = 'Fall' and year =
    2009)
except
```

Set Operations (2)

- Find the salaries of all instructors that are less than the largest salary

```
select distinct T.salary
from instructor as T, instructor as S
where T.salary < S.salary;
```

- Find all the salaries of all instructors

```
select distinct salary
from instructor;
```

- Find the largest salary of all instructors

```
(select 'second query')
except
(select 'first query');
```

Set Operations (3)

Duplicates

- Set operations `union`, `intersect`, and `except`

Set Operations (3)

Duplicates

- Set operations `union`, `intersect`, and `except`
- Each of the above operations automatically eliminates duplicates

Set Operations (3)

Duplicates

- Set operations `union`, `intersect`, and `except`
- Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.

Set Operations (3)

Duplicates

- Set operations `union`, `intersect`, and `except`
- Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- Suppose a tuple occurs m times in r and n times in s, then, it occurs:

Set Operations (3)

Duplicates

- Set operations `union`, `intersect`, and `except`
- Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- Suppose a tuple occurs m times in r and n times in s, then, it occurs:
  - $m + n$ times in r `union all` s

Set Operations (3)

Duplicates

- Set operations `union`, `intersect`, and `except`
- Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- Suppose a tuple occurs m times in r and n times in s, then, it occurs:
  - $m + n$ times in r `union all` s
  - $\min(m, n)$ times in r `intersect all` s

Set Operations (3)

Duplicates

- Set operations `union`, `intersect`, and `except`
- Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- Suppose a tuple occurs m times in r and n times in s, then, it occurs:
  - $m + n$ times in r `union all` s
  - $\min(m, n)$ times in r `intersect all` s
  - $\max(0, m - n)$ times in r `except all` s

Null Values

Null Values

- It is possible for tuples to have a null value, denoted by `null`, for some of their attributes
- `null` signifies an unknown value or that a value does not exist
- The result of any arithmetic expression involving `null` is `null`
  - Example: `5 + null` returns `null`
- The predicate `is null` can be used to check for null values
  - Example: Find all instructors whose salary is null

```
select name
from instructor
where salary is null;
```

- It is not possible to test for null values with comparison operators, such as =, <, or <>. We need to use the `is null` and `is not null` operators instead

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**
  - Example: `5 < null` or `null <> null` or `null = null`

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**
  - Example: `5 < null` or `null <> null` or `null = null`
- Three-valued logic using the value **unknown**:

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**
  - Example: `5 < null` or `null <> null` or `null = null`
- Three-valued logic using the value **unknown**:
  - **OR**: `(unknown or true) = true`, `(unknown or false) = unknown`, `(unknown or unknown) = unknown`

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**
  - Example: `5 < null` or `null <> null` or `null = null`
- Three-valued logic using the value **unknown**:
  - **OR**: `(unknown or true) = true`, `(unknown or false) = unknown`, `(unknown or unknown) = unknown`
  - **AND**: `(true and unknown) = unknown`, `(false and unknown) = false`, `(unknown and unknown) = unknown`

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**
  - Example: `5 < null` or `null <> null` or `null = null`
- Three-valued logic using the value **unknown**:
  - **OR**: `(unknown or true) = true`, `(unknown or false) = unknown`, `(unknown or unknown) = unknown`
  - **AND**: `(true and unknown) = unknown`, `(false and unknown) = false`, `(unknown and unknown) = unknown`
  - **NOT**: `(not unknown) = unknown`

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**
  - Example: `5 < null` or `null <> null` or `null = null`
- Three-valued logic using the value **unknown**:
  - **OR**: `(unknown or true) = true`, `(unknown or false) = unknown`, `(unknown or unknown) = unknown`
  - **AND**: `(true and unknown) = unknown`, `(false and unknown) = false`, `(unknown and unknown) = unknown`
  - **NOT**: `(not unknown) = unknown`
- '`P is unknown`' evaluates to **true** if predicate P evaluates to **unknown**

Null Values (2): Three Valued Logic

Overview

- Three values: **true**, **false**, **unknown**
- Any comparison with null returns **unknown**
  - Example: `5 < null` or `null <> null` or `null = null`
- Three-valued logic using the value **unknown**:
  - **OR**: `(unknown or true) = true`, `(unknown or false) = unknown`, `(unknown or unknown) = unknown`
  - **AND**: `(true and unknown) = unknown`, `(false and unknown) = false`, `(unknown and unknown) = unknown`
  - **NOT**: `(not unknown) = unknown`
- '`P is unknown`' evaluates to **true** if predicate P evaluates to **unknown**
- Result of `where` clause predicate is treated as **false** if it evaluates to **unknown**

Aggregate Functions

Aggregate Functions

Common Functions

- These functions operate on the multiset of values of a column of a relation, and return a value

Aggregate Functions

Common Functions

- These functions operate on the multiset of values of a column of a relation, and return a value
  - avg: average value

Aggregate Functions

Common Functions

- These functions operate on the multiset of values of a column of a relation, and return a value
  - `avg`: average value
  - `min`: minimum value

Aggregate Functions

Common Functions

- These functions operate on the multiset of values of a column of a relation, and return a value
  - `avg`: average value
  - `min`: minimum value
  - `max`: maximum value

Aggregate Functions

Common Functions

- These functions operate on the multiset of values of a column of a relation, and return a value
  - `avg`: average value
  - `min`: minimum value
  - `max`: maximum value
  - `sum`: sum of values

Aggregate Functions

Common Functions

- These functions operate on the multiset of values of a column of a relation, and return a value
  - `avg`: average value
  - `min`: minimum value
  - `max`: maximum value
  - `sum`: sum of values
  - `count`: number of values

Aggregate Functions (2)

- Find the average salary of instructors in the Computer Science department

```
select avg(salary)
from instructor
where dept_name = 'Comp. Sci';
```

- Find the total number of instructors who teach a course in the Spring 2010 semester

```
select count(distinct ID)
from teaches
where semester = 'Spring' and year = 2010;
```

- Find the number of tuples in the course relation

```
select count(*)
from course;
```

Aggregate Functions (3): Group By

- Find the average salary of instructors in each department

```
select dept_name, avg(salary) as avg_salary
from instructor
group by dept_name;
```

instructor relation

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
76543	Singh	Finance	80000
...	...	...	...
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

Result

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Finance	85000
...	...
Physics	91000

Aggregate Functions (4): Group By

- Attributes in select clause outside of aggregate functions must appear in group by list

```
/* erroneous query */
select dept_name, ID, avg(salary)
from instructor
group by dept_name;
```

Aggregate Functions (5): Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name, avg(salary)
from instructor
group by dept_name
having avg(salary) > 42000;
```

- **Note**: Predicates in the `having` clause are applied **after** the formation of groups, whereas predicates in the `where` clause are applied **before** forming groups.

Null Values and Aggregates

- Total all salaries

```
select sum(salary)
from instructor;
```

Null Values and Aggregates

- Total all salaries

```
select sum(salary)
from instructor;
```

- Above statement ignores null amounts

Null Values and Aggregates

- Total all salaries

```
select sum(salary)
from instructor;
```

- Above statement ignores null amounts
- Result is null if there is no non-null amount

Null Values and Aggregates

- Total all salaries

```
select sum(salary)
from instructor;
```

- Above statement ignores null amounts
- Result is null if there is no non-null amount
- All aggregate operations except `count(*)` ignore tuples with null values on the aggregated attributes

Null Values and Aggregates

- Total all salaries

```
select sum(salary)
from instructor;
```

- Above statement ignores null amounts
- Result is null if there is no non-null amount
- All aggregate operations except `count(*)` ignore tuples with null values on the aggregated attributes
- What if collection has only null values?

Null Values and Aggregates

- Total all salaries

```
select sum(salary)
from instructor;
```

- Above statement ignores null amounts
- Result is null if there is no non-null amount
- All aggregate operations except `count(*)` ignore tuples with null values on the aggregated attributes
- What if collection has only null values?
  - `count` returns 0

Null Values and Aggregates

- Total all salaries

```
select sum(salary)
from instructor;
```

- Above statement ignores null amounts
- Result is null if there is no non-null amount
- All aggregate operations except `count(*)` ignore tuples with null values on the aggregated attributes
- What if collection has only null values?
  - `count` returns 0
  - all other aggregates return `null`

Module Summary

- Completed the understanding of set operations, null values, and aggregation

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.